

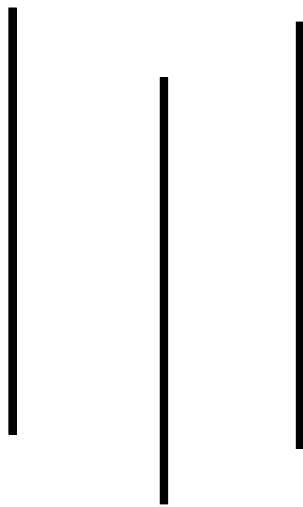
Multiple View Geometry

Lab3 – Epipolar Geometry and Stereo

Lab Sessions: 2 (4 hours)

Total Work Load: 10 hours

Programming Language: MATLAB



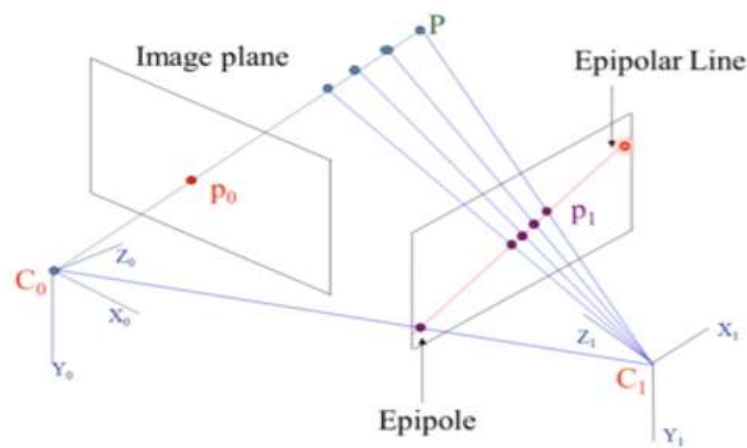
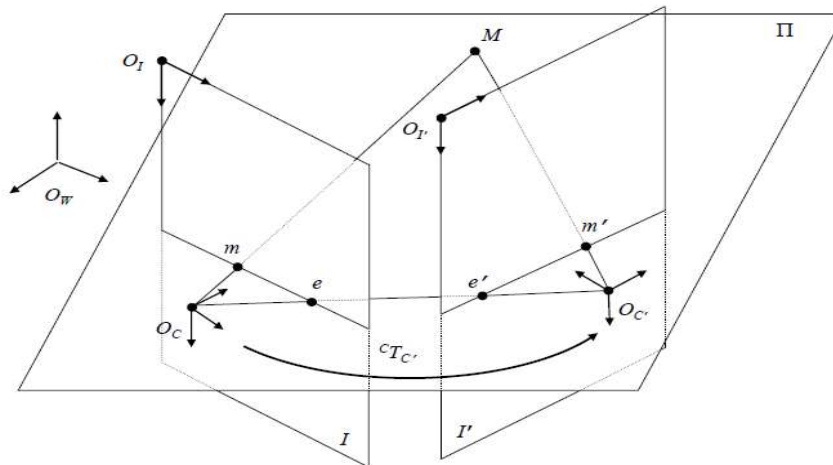
Submitted by

1. **Pravin Oli** (u1999699) [*pravin.oli.08@gmail.com*]
2. **Gebrecherkos Gebrezgabhier** (u1999542) [*chereg2016@gmail.com*]

Table of Contents

Topic_1 Epipolar Geometry:	3
Definitions of Key Terms:	3
Mathematical modelling:	4
Topic_2 Fundamental Matrix:	5
1.Objectives:	5
2.Methodology:.....	6
Step-1 (Camera-1 Parameters).....	6
Step-2 (Camera-2 Parameters).....	6
Step-3 (Camera projection matrices)	6
Step-4 (Fundamental matrix analytically)	7
Step-5 (Define set of points).....	8
Step-6 (Compute the pairs of image points)	9
Step-7 (Plot the 2D points obtained in step 6)	9
Step-8 (fundamental matrix with the 8-point method)	10
Step-9 (sum of absolute differences)	11
Step-10 (Epoles and Epipolar lines)	11
Step-11 (Add gaussian Noise on 2D points).....	12
Step-12 (Computation with Noisy 2D points)	13
Step-13 (Gaussian noise increase in the range $[-2, +2]$)	16
Step-14 (Coordinate normalization method)	17
3. Conclusion	19
4. Appendix (Code).....	19

Topic_1 Epipolar Geometry:



Definitions of Key Terms:

1. Epipolar Geometry:

Epipolar geometry is the intrinsic projective geometry between two 2D views of the same 3D scene captured by two cameras. It describes the relationship between the 3D points in the scene and their projections onto the image planes of the two cameras. It forms the foundation of stereo vision, enabling 3D reconstruction and depth estimation.

The main components include:

- Epipolar planes,
- Epipolar lines,
- Epipoles.

2. Epipolar Plane:

The epipolar plane is the 3D plane that passes through two camera centres (O_C) and ($O_{C'}$) and a 3D point M in the scene. It intersects the image planes of the two cameras, forming corresponding epipolar lines. Every 3D point in the scene has its own epipolar plane.

3. Epipolar Lines:

An epipolar line is the intersection of the epipolar plane with an image plane. Epipolar lines always pass through the epipole in the respective image.

- For a point m in one image, the corresponding epipolar line in the other image is the locus of all possible points m' that the 3D point M could project to in the second view.

4. Epipoles:

Epipoles are the points where the line connecting the two camera centres ($OC \rightarrow OC'$) intersects the image planes. All epipolar lines in an image pass through the epipole.

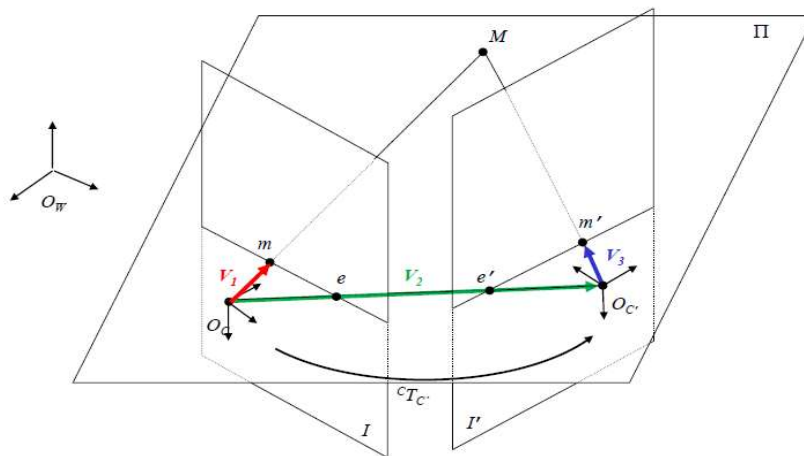
- Epipole in Image 1:** The projection of the second camera centre (OC') onto the first image plane.
- Epipole in Image 2:** The projection of the first camera centre (OC) onto the second image plane.

5. Correspondence Points:

Correspondence points are pairs of points in the two images that are projections of the same 3D point in the scene.

- If a 3D point M is observed by two cameras, its projections are m in the first image and m' in the second image. These two points are called corresponding points.

Mathematical modelling:



Vectors V_1 , V_2 and V_3 are co-planar
We can express this as $V_1 \cdot (V_2 \times V_3) = 0$

Since V_1 , V_2 and V_3 are on same plane so the cross product of two vector is perpendicular with the remaining vector hence dot product becomes zero.

$$V_1 = m = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$V_2 = t = \begin{bmatrix} tx \\ ty \\ tz \end{bmatrix}$$

$$V_3 = R \cdot m' = \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix}$$

$$\text{Then, } (V_2 \times V_3) = (t \times R \cdot m') = \begin{bmatrix} tx \\ ty \\ tz \end{bmatrix} \times \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} ty \cdot z' - tz \cdot y' \\ tx \cdot z' - tz \cdot x' \\ tx \cdot y' - ty \cdot x' \end{bmatrix} = \begin{bmatrix} 0 & -tz & ty \\ tz & 0 & -tx \\ -ty & tx & 0 \end{bmatrix} \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix}$$

$$V1. (V2 \times V3) = m. (t \times R.m') = \begin{bmatrix} x \\ y \\ z \end{bmatrix}^T \begin{bmatrix} 0 & -tz & ty \\ tz & 0 & -tx \\ -ty & tx & 0 \end{bmatrix} \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix}$$

Or we can express it as:

$$V1. (V2 \times V3) = m^T. [t]_X. Rm' = 0$$

Also:

$$m^T. [t]_X. Rm' = m^T. E. m'$$

Where E is essential matrix

$$E = [t]_X. R$$

The $[t_x]$ matrix is skew-symmetric matrix and R is rotation matrix which is orthonormal hence it is possible to decouple Tx and R from their product using singular value decomposition method.

Topic_2 Fundamental Matrix:

In above equation we don't know the value of m and m' coordinates so we use intrinsic camera parameter to compute fundamental matrix.

$$\begin{aligned} \tilde{m} &= Km & m &= K^{-1}\tilde{m} \\ \tilde{m}' &= K'm' & m' &= K'^{-1}\tilde{m}' \end{aligned} \quad K = \begin{pmatrix} \alpha_u & 0 & u_0 \\ 0 & \alpha_v & v_0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$m^T E m' = 0$$

$$(K^{-1}\tilde{m})^T E K'^{-1}\tilde{m}' = 0$$

$$\tilde{m}^T \underbrace{K^{-T} E K'^{-1}}_F \tilde{m}' = 0$$

$$\boxed{\tilde{m}^T F \tilde{m}' = 0}$$

The 3x3 matrix F is now our Fundamental Matrix.

$$\text{Hence } F = K^{-T} \times E \times K^{-T}$$

We know value of K^{-T} , E, K'^{-T} . we can now compute fundamental matrix F

1.Objectives:

- To Construct and analyse the Fundamental matrix for a stereo camera setup using both analytical methods and the 8-point algorithm, comparing the results.
- Visualize the epipolar geometry by plotting points, epipoles, and epipolar lines in the image planes.
- Evaluate the robustness of the epipolar geometry by introducing noise to 2D point projections and analysing its effects.

2.Methodology:

Step-1 (Camera-1 Parameters)

Implementation:

At first camera 1 parameters were defined and world coordinate system was set to coordinate system of camera 1.

Code:

```
%% Step 1
% Camera 1
au1 = 100; av1 = 120; uo1 = 128; vo1 = 128;
imageSize = [256 256];
```

Step-2 (Camera-2 Parameters)

Implementation:

Now camera 2 parameters were defined w.r.t coordinate system of camera 1.

Code:

```
%% Step 2
% Camera 2
au2 = 90; av2 = 110; uo2 = 128; vo2 = 128;
ax = 0.1; by = pi/4; cz = 0.2; % XYZ EULER
tx = -1000; ty = 190; tz = 230;
```

Step-3 (Camera projection matrices)

Implementation:

In this step intrinsic matrix for both camera (1 and 2) is computed with rotation and translation between both cameras. Camera projection matrix P1 and P2 are also calculated.

Code:

```
%% STEP 3

% Compute intrinsic matrices and projection matrices

K1 = [au1 0 uo1; 0 av1 vo1; 0 0 1]; % Intrinsics matrix for camera 1

wR1c = eye(3); % rotation of camera 1, from the camera to the world
coordinate frame

wt1c = [0 0 0]'; % translation of camera 1, from the camera to the world
coordinate frame

K2 = [au2 0 uo2; 0 av2 vo2; 0 0 1]; % Intrinsics matrix for camera 2

rot_x=[1, 0, 0; 0, cos(ax), -sin(ax);0, sin(ax), cos(ax)];

rot_y=[cos(by), 0, sin(by); 0, 1, 0;-sin(by), 0, cos(by)];

rot_z=[cos(cz), -sin(cz), 0; sin(cz), cos(cz), 0;0, 0, 1];

wR2c = rot_x*rot_y*rot_z % rotation of camera 2, from the camera to the world
coordinate frame
```

```
wt2c = [tx; ty; tz] % translation of camera 2, from the camera to the world
coordinate frame
```

```
P1 = K1 * [wR1c' -wR1c' * wt1c];
```

```
P2 = K2 * [wR2c' -wR2c' * wt2c]; % You have to replace this. This is just to
serve as input for the drawing functions
```

Output:

```
wR2c =
    0.6930    -0.1405     0.7071
    0.2669     0.9611    -0.0706
   -0.6697     0.2376     0.7036
wt2c =
   -1000
    190
    230
Camera projection 1
    100     0    128     0
     0    120    128     0
     0     0     1     0
Camera projection 1
1.0e+05 *
    0.0015    0.0001    0.0003    1.4318
    0.0008    0.0010    0.0012    0.2996
    0.0000   -0.0000    0.0000    0.0056
```

Step-4 (Fundamental matrix analytically)

Implementation:

In this step fundamental matrix is calculated. It is the product of intrinsic matrices with rotation and translation. The translation matrix is the form of its skew-symmetric matrix.

$$F_{1 \leftrightarrow 2} = K_2^{-T} \cdot R_{2 \leftrightarrow 1}^T \cdot [t_{2 \leftrightarrow 1}]_X \cdot K_1^{-1}$$

Code:

```
%% STEP 4

% Attention: This is an invented matrix just to have some input for the
drawing
% functions. You have to compute it properly
F = [3e-05 7e-05 -0.006; 2e-05 -2.e-05 0.01; -0.009 -0.01 1];

F = F./F(3,3);
fprintf('Step 4:\n\tAnalytically obtained F:\n');
disp(F);

% Step 1: Compute translation vector from Camera 1 to Camera 2
t2to1 = wt2c - wt1c; % Translation vector from camera 1 to camera 2
tx = t2to1(1);
ty = t2to1(2);
tz = t2to1(3);
```

```
% Step 2: Construct the skew-symmetric matrix [t2to1]_X
t2to1_X = [0, -tz, ty;
           tz, 0, -tx;
           -ty, tx, 0];

% Step 3: Compute the rotation matrix R2to1 (this is the transpose of wR2c)
R2to1 = wR2c';

% Step 4: Compute the Fundamental matrix Flto2 using the formula
Flto2 = K2 \ (R2to1 * t2to1_X) * K1^-1;

% Display the result
disp('Fundamental Matrix Flto2:');
disp(Flto2);
```

Output:

```
Step 4:
Analytically obtained F:
    0.0000    0.0001   -0.0060
    0.0000   -0.0000    0.0100
   -0.0090   -0.0100    1.0000
Fundamental Matrix Flto2:
 1.0e+03 *
    0.0022    0.0103   -1.6820
    0.0018    0.0084   -1.3642
   -0.0015   -0.0072    1.1796
```

Step-5 (Define set of points)

Implementation:

In this step we defined set of object points provided, with respect to the world coordinate system (same as the camera 1 coordinate system).

```
V(:,1)=[100; -400; 2000];
V(:,2)=[300; -400; 3000];
V(:,3)=[500; -400; 4000];
V(:,4)=[700; -400; 2000];
V(:,5)=[900; -400; 3000];
V(:,6)=[100; -40; 4000];
V(:,7)=[300; -40; 2000];
V(:,8)=[500; -40; 3000];
V(:,9)=[700; -40; 4000];
V(:,10)=[900; -40; 2000];
V(:,11)=[100; 40; 3000];
V(:,12)=[300; 40; 4000];
V(:,13)=[500; 40; 2000];
V(:,14)=[700; 40; 3000];
V(:,15)=[900; 40; 4000];
V(:,16)=[100; 400; 2000];
V(:,17)=[300; 400; 3000];
V(:,18)=[500; 400; 4000];
V(:,19)=[700; 400; 2000];
V(:,20)=[900; 400; 3000];
```

Step-6 (Compute the pairs of image points)

Implementation:

In this step we computed the pairs of image points in both image planes by using the camera projection matrices of step 3.

Code:

```
%% STEP 6
% Projection on image planes
cam1_p2d = mvg_projectPointToImagePlane(V,P1);
cam2_p2d = mvg_projectPointToImagePlane(V,P2);
```

Output:

```
pairs of image points in P1
Columns 1 through 12

133.0000 138.0000 140.5000 163.0000 158.0000 130.5000 143.0000
144.6667 145.5000 173.0000 131.3333 135.5000
104.0000 112.0000 116.0000 104.0000 112.0000 126.8000 125.6000
126.4000 126.8000 125.6000 129.6000 129.2000

Columns 13 through 20

153.0000 151.3333 150.5000 133.0000 138.0000 140.5000 163.0000
158.0000
130.4000 129.6000 129.2000 152.0000 144.0000 140.0000 152.0000
144.0000

pairs of image points in P2
Columns 1 through 12

133.0000 138.0000 140.5000 163.0000 158.0000 130.5000 143.0000
144.6667 145.5000 173.0000 131.3333 135.5000
104.0000 112.0000 116.0000 104.0000 112.0000 126.8000 125.6000
126.4000 126.8000 125.6000 129.6000 129.2000

Columns 13 through 20

153.0000 151.3333 150.5000 133.0000 138.0000 140.5000 163.0000
158.0000
130.4000 129.6000 129.2000 152.0000 144.0000 140.0000 152.0000
144.0000
```

Step-7 (Plot the 2D points obtained in step 6)

Implementation:

The output value from step 6 was plotted.

Code:

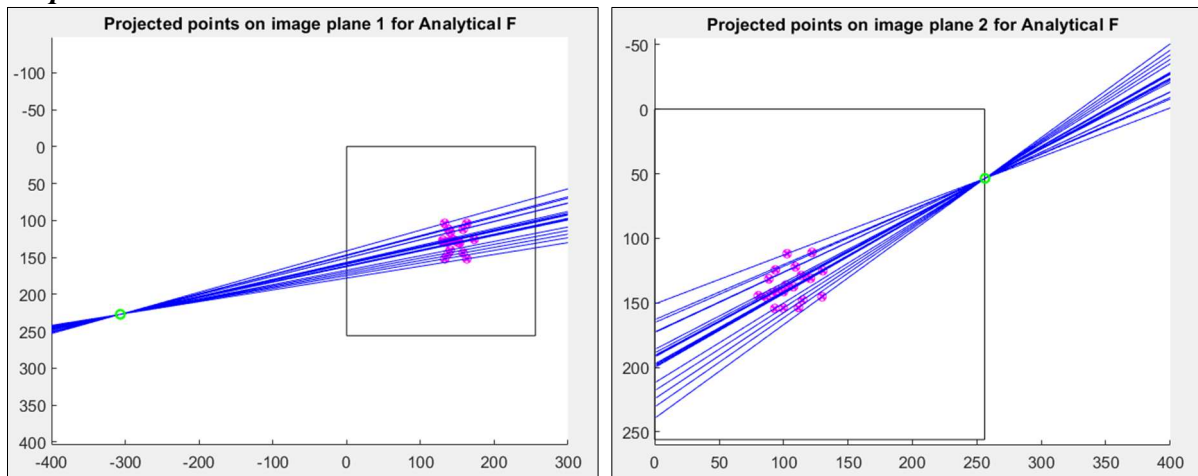
```
%% STEP 7
% example of the plotting functions
% Draw 2D projections on image planes
cam1_fig = mvg_show_projected_points(cam1_p2d(1:2,:),imageSize,'Projected
points on image plane 1 for Analytical F');
cam2_fig = mvg_show_projected_points(cam2_p2d(1:2,:),imageSize,'Projected
points on image plane 2 for Analytical F');

% Draw epipolar lines
```

```
[~,~,c1_l_coeff,c2_l_coeff] =
mvg_compute_epipolar_geom_modif(cam1_p2d,cam2_p2d,F);
[cam1_fig,cam2_fig] = mvg_show_epipolar_lines(cam1_fig, cam2_fig,
c1_l_coeff,c2_l_coeff, [-400,1;300,400],'b');

% Draw epipoles
% Draw epipoles Using Null space
e1 = null(F);
e2 = null(F');
% Normalize
ep_1 = e1/e1(3);
ep_2 = e2/e2(3);
[~,~] = mvg_show_epipoles(cam1_fig, cam2_fig,ep_1,ep_2);
```

Output:



Step-8 (fundamental matrix with the 8-point method)

Implementation:

In this step, here we calculated the fundamental matrix with the 8-point method and SVD using of the 2D points obtained in step 6.

Code:

```
%% Step8
%Fundamental matrix with 8-points
enforce_rank2 = 0;
New_F = compute_fundamental_m(cam1_p2d,cam2_p2d);
New_F = New_F./New_F(3,3);%normalizing
fprintf('Fundamental_with_8_point:Step8:\n');
disp(New_F);
```

Output:

```
Fundamental_with_8_point: Step8:
    0.0000    0.0001   -0.0066
    0.0000   -0.0000    0.0130
   -0.0096   -0.0173    1.0000
```

Step-9 (sum of absolute differences)

Implementation:

In this step the difference in value of fundamental matrix with the 8-point method (step-8) with analytical value of fundamental matrix (step-4) is calculated in terms of the sum of absolute differences.

Code:

```
%% Step9
% sum of absolute differences
sad= sum(abs(F(:) - New_F(:)));
disp(['Sum of Absolute Differences of F ana New_F: ', num2str(sad)]);
```

Output:

```
Sum of Absolute Differences of F ana New_F: 1.667e-15
```

Step-10 (Epipoles and Epipolar lines)

Implementation:

In this step the epipoles and epipolar lines are drawn in the image points plotted in step 7 using the value of fundamental matrix with the 8-point method (step-8).

For each point $[u_1, v_1, 1]^T$ in image 1, the corresponding epipolar line in image 2 is computed using the Fundamental Matrix F:

$$l' = [l_1, l_2, l_3] = F \times [u_1, v_1, 1]^T$$

The line equation in image 2 is given by:

$$l_1 \times u_2 + l_2 \times v_2 + l_3 = 0$$

Rearranging to express v_2 in terms of u_2 as:

$$v_2 = -\frac{l_1}{l_2}u_2 - \frac{l_3}{l_2}$$

Where:

- $m = -\frac{l_1}{l_2}$ (slope)
- $d = -\frac{l_3}{l_2}$ (intercept)

Code:

```
%% Step10
% Display projected points on image planes
img1_proj = mvg_show_projected_points(cam1_p2d(1:2, :), imageSize,
'Projected points on Image Plane 1 (8-Point, STEP10)');
img2_proj = mvg_show_projected_points(cam2_p2d(1:2, :), imageSize,
'Projected points on Image Plane 2 (8-Point, STEP10)');

% Compute and draw epipolar lines
[~, ~, epiline_coeffs_img1, epiline_coeffs_img2] =
mvg_compute_epipolar_geom_modif(cam1_p2d, cam2_p2d, New_F);
[img1_with_lines, img2_with_lines] = mvg_show_epipolar_lines(img1_proj,
img2_proj, epiline_coeffs_img1, epiline_coeffs_img2, [-400, 1; 300, 400],
'k');

% Compute epipoles
epipole_img1 = null(New_F);
```

```

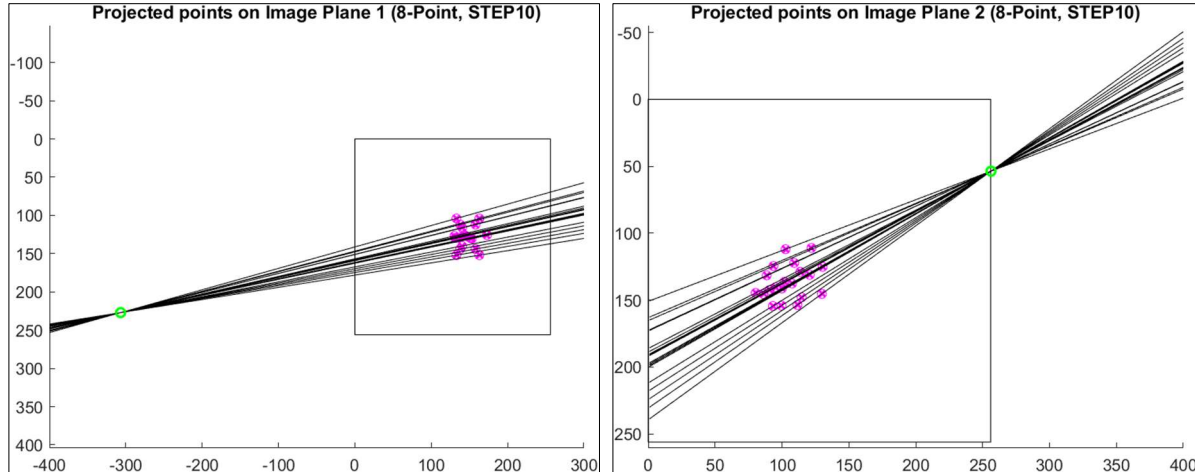
epipole_img2 = null(New_F');

% Normalize epipoles
epipole_img1 = epipole_img1 / epipole_img1(3);
epipole_img2 = epipole_img2 / epipole_img2(3);

% Show epipoles on images
[~, ~] = mvz_show_epipoles(img1_with_lines, img2_with_lines, epipole_img1,
epipole_img2);

```

Output:



Step-11 (Add gaussian Noise on 2D points)

Implementation:

In this step zero-mean Gaussian noise was added to the 2D points such that 95% of that noise will be the range $[-1, +1]$ pixels.

Code:

```

%% Step 11: Adding noise to points
% standard deviation for noise
noise_std_dev = 0.5;

% Generate random noise with normal distribution
cam1_points_noisy = cam1_p2d + normrnd(0, noise_std_dev, size(cam1_p2d));
cam2_points_noisy = cam2_p2d + normrnd(0, noise_std_dev, size(cam2_p2d));

% Displaying the original and noisy points for comparison
fprintf('Step 11: Noise-free points (Camera 1):\n');
disp(cam1_p2d);

fprintf('Step 11: Noisy points (Camera 1):\n');
disp(cam1_points_noisy);

```

Output:

```

Step 11: Noise-free points (Camera 1):
Columns 1 through 12

    133.0000    138.0000    140.5000    163.0000    158.0000    130.5000    143.0000
    144.6667    145.5000    173.0000    131.3333    135.5000
    104.0000    112.0000    116.0000    104.0000    112.0000    126.8000    125.6000
    126.4000    126.8000    125.6000    129.6000    129.2000

```

Columns 13 through 20						
153.0000	151.3333	150.5000	133.0000	138.0000	140.5000	163.0000
158.0000						
130.4000	129.6000	129.2000	152.0000	144.0000	140.0000	152.0000
144.0000						
Step 11: Noisy points (Camera 1):						
Columns 1 through 12						
133.2688	136.8706	140.6594	162.7832	159.7892	129.8251	143.3627
145.0240	145.4379	173.7045	131.6691	135.8586		
104.9169	112.4311	115.3462	104.1713	113.3847	128.3175	125.5685
126.2975	127.5448	126.3086	128.9963	130.0151		
Columns 13 through 20						
153.2444	151.6968	150.6469	133.4442	137.4656	139.0279	163.1626
158.6851						
130.9173	129.4483	128.8064	151.4265	143.5953	140.7192	151.6225
143.1442						

Step-12 (Computation with Noisy 2D points)

Implementation:

In this step the process from step 8 up to 10 was repeated with the noisy 2D points.

Questions: Compare the epipolar geometry obtained.

-Are points on the epipolar lines?

- No, for noisy 2D points they are not lying on epipolar lines (refer figure1 below). This happens because the noise in the data introduces inconsistencies, and the Fundamental matrix (F) does not accurately represent the epipolar geometry

-Are the epipolar lines all converging on a single point?

- No, for implementation without rank, the epipolar lines are not converging on epipole (refer figure2 below). They are shifted by some distance above epipoles, due to the incorrect structure of the F matrix caused by noise.
- Yes, for implementation with rank, epipolar lines are converging on single point which is epipoles (refer figure 3 and figure 4 below), because enforcing the rank constraint ensures F is valid for epipolar geometry.

-What condition should the F matrix fulfil so that all epipolar lines cross at the same point?

- For all epipolar lines to intersect at a single point (the epipole), the **fundamental matrix** F must be of **rank 2**. This condition ensures that F represents a valid relationship between corresponding points in the two images.

Impose that condition:

-Verify if all epipolar lines now cross at single same point on each image.

- Perform Singular Value Decomposition (SVD) of F :
- $F = U \cdot \text{diag}(\sigma_1, \sigma_2, \sigma_3) \cdot V^T$
where $\sigma_1, \sigma_2, \sigma_3$ are the singular values.
- Modify F by setting the smallest singular value (σ_3) to zero:
- $F' = U \cdot \text{diag}(\sigma_1, \sigma_2, 0) \cdot V^T$
- This ensures F' has rank 2.

- Recompute the epipolar lines using the updated F' .
- Verify that all epipolar lines now converge at a single point in both images.

-Investigate how the location of both epipoles can be computed directly from the F matrix, using the SVD, and describe the procedure.

- **Epipole in Image 1 (e_1):** Solve $F^T \cdot e_1 = 0$, where e_1 is the null vector of F^T .
- **Epipole in Image 2 (e_2):** Solve $F \cdot e_2 = 0$, where e_2 is the null vector of F .
- **Procedure:**

Perform SVD of F :

$$F = U\Sigma V^T$$

The epipole in Image 1 (e_1) is the last column of V .

The epipole in Image 2 (e_2) is the last column of U .

- **Normalization:** Normalize e_1 and e_2 to ensure the third coordinate is 1 (homogeneous form).

The code for enforcing the rank is in the `compute_fundamental_m` script.

We call it inside: `Noisy_F12 = compute_fundamental_m`

Code:

```
%% Step12
%Step8 with noisy points
% Fundamental with 8 point for noisy Points
enforce_rank2 = 0;
Noisy_F12 = compute_fundamental_m(cam1_points_noisy,cam2_points_noisy);
% normalize all the points by
Noisy_F12 = Noisy_F12./Noisy_F12(end, end);
fprintf('Step 12: Fundamental_M_for_noise_point:\n');
disp(Noisy_F12);

%Step9 for noisy
% error difference with noise
% sum of absolute differences (SAD)
sad_noisy = sum(abs(F(:) - Noisy_F12(:)));
disp(['Sum of Absolute Differences of F and Noisy_F: ',
num2str(sad_noisy)]);

%Step10 for noisy
% Drawing Epipolar lines and epipole for noised one
cam1_fig =
mvg_show_projected_points(cam1_points_noisy(1:2,:),imageSize,'Projected
points on image plane 1 for noisy Step12');
cam2_fig =
mvg_show_projected_points(cam2_points_noisy(1:2,:),imageSize,'Projected
points on image plane 2 for noisy Step12');

% Draw epipolar lines
[~,~,c1_l_coeff_s12_noise,c2_l_coeff_s12_noise] =
mvg_compute_epipolar_geom_modif(cam1_points_noisy,cam2_points_noisy,Noisy_F
12);
```

```
[cam1_fig,cam2_fig] = mvg_show_epipolar_lines(cam1_fig, cam2_fig,
c1_l_coeff_s12_noise,c2_l_coeff_s12_noise, [-400,1;300,400],'c');

% Draw epipoles
% Epipoles Investigation with SVD
[U,~, V] = svd(Noisy_F12);
ep1_12_noise= V(:, end);
ep2_12_noise= U(:, end);

% Normalization
ep1_12_noise = ep1_12_noise / ep1_12_noise(3);
ep2_12_noise = ep2_12_noise / ep2_12_noise(3);

[~,~] = mvg_show_epipoles(cam1_fig, cam2_fig,ep1_12_noise,ep2_12_noise);
```

Output

Step 12: Fundamental_M_for_noisy_point:

0.0000	0.0000	-0.0027
0.0000	0.0000	-0.0021
-0.0073	-0.0038	1.0000

Sum of Absolute Differences of F ana Noisy_F: 0.034872

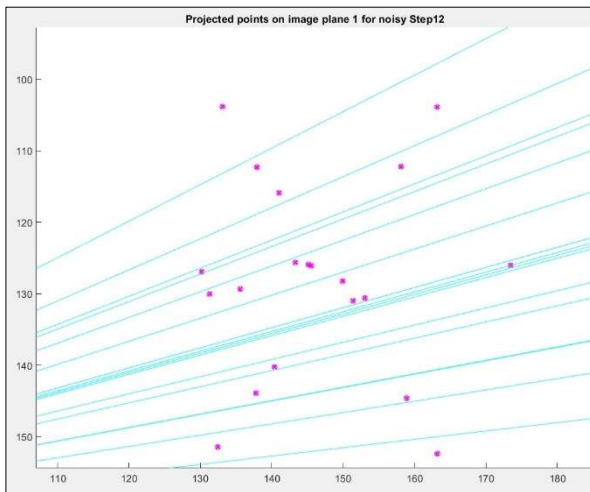


Figure1 points not lying on epipolar lines (Zoom in image)

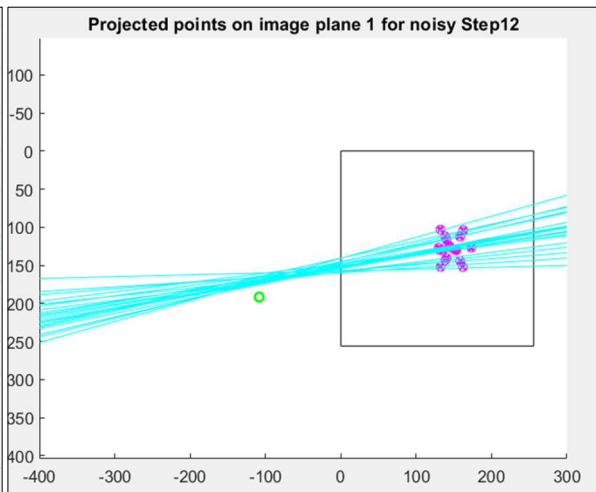


Figure2 convergence without rank (not on epipole)

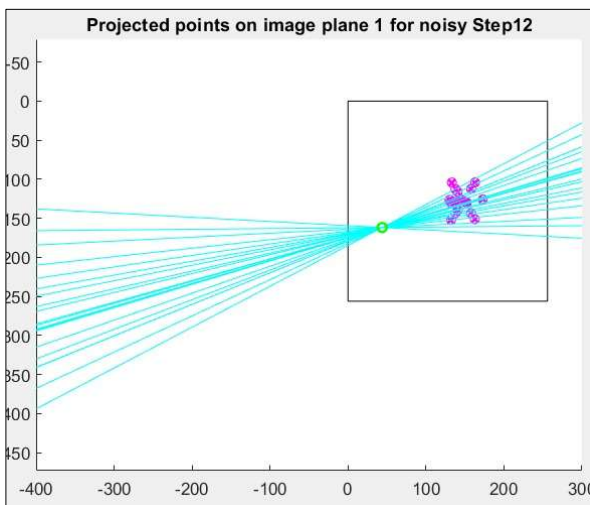


Figure3: Rank enforcement plane 1

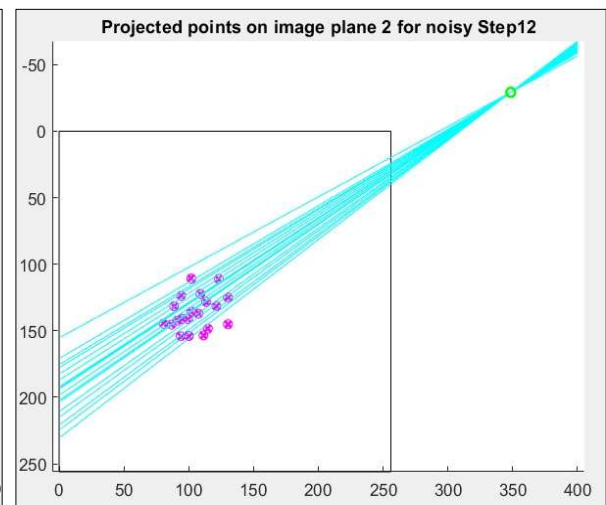


Figure4: Rank enforcement plane 2

Step-13 (Gaussian noise increase in the range [-2, +2])

Implementation:

In this step the process from step 8 up to 12 was repeated with the increment in noise in the range [-2, +2] for 95% of points)

Code:

```
%% Step 13: Adding Noise and Recomputing

% Add Gaussian noise to points (std = 1 for 95% of values in [-2, +2])
noise_std_dev = 1;
cam1_points_noisy_13 = cam1_p2d + normrnd(0, noise_std_dev,
size(cam1_p2d));
cam2_points_noisy_13 = cam2_p2d + normrnd(0, noise_std_dev,
size(cam2_p2d));

% Compute fundamental matrix with noisy points and normalize
Noisy_F13 = compute_fundamental_m(cam1_points_noisy_13,
cam2_points_noisy_13);
Noisy_F13 = Noisy_F13 ./ Noisy_F13(end, end);
fprintf('Step 13: Fundamental Matrix with Noisy Points:\n');
disp(Noisy_F13);

% Compute error (Sum of Absolute Differences) between F and noisy F
sad_noisy_13 = sum(abs(F(:) - Noisy_F13(:)));
disp(['Sum of Absolute Differences (F vs Noisy_F13): ',
num2str(sad_noisy_13)]);

% Visualize noisy points projection on image planes
cam1_fig = mvg_show_projected_points(cam1_points_noisy_13(1:2, :),
imageSize, 'Image 1: Noisy Points Step13');
cam2_fig = mvg_show_projected_points(cam2_points_noisy_13(1:2, :),
imageSize, 'Image 2: Noisy Points Step13');

% Compute and draw epipolar lines with noisy points
[~, ~, c1_l_coeff_s13_noise, c2_l_coeff_s13_noise] =
mvg_compute_epipolar_geom_modif(cam1_points_noisy_13, cam2_points_noisy_13,
Noisy_F13);
[cam1_fig, cam2_fig] = mvg_show_epipolar_lines(cam1_fig, cam2_fig,
c1_l_coeff_s13_noise, c2_l_coeff_s13_noise, [-400, 1; 300, 400], 'r');

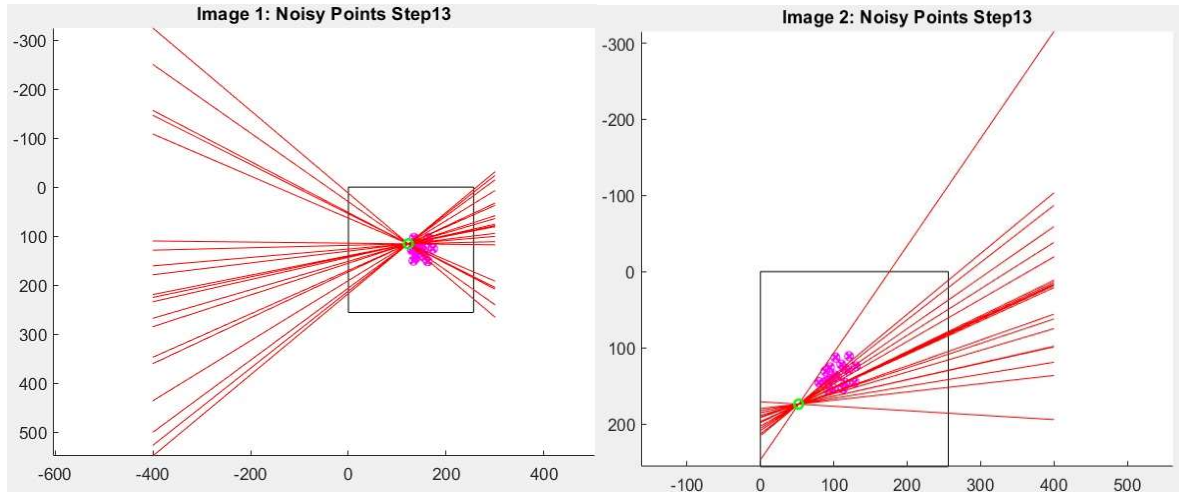
% Compute epipoles using SVD and normalize
[U, ~, V] = svd(Noisy_F13);
ep1_13_noise = V(:, end) / V(3, end);
ep2_13_noise = U(:, end) / U(3, end);

% Display epipoles on images
[~, ~] = mvg_show_epipoles(cam1_fig, cam2_fig, ep1_13_noise, ep2_13_noise);
```

Output:

```
Step 13: Fundamental Matrix with Noisy Points:
    0.0000    -0.0000    -0.0021
    0.0000     0.0000    -0.0065
   -0.0048    -0.0014     1.0000

Sum of Absolute Differences (F vs Noisy_F13): 0.04487
```



Step-14 (Coordinate normalization method)

Implementation:

In this step, we normalized the coordinates of the projected points in both images using a coordinate normalization method. This involves applying transformations T and T' (homographies) that adjust the points so that they are centred at the origin and have a standard deviation of 1.

We then repeated steps 8-12, which include computing the Fundamental matrix using the normalized points, adding Gaussian noise within the range $[-2, +2]$ for 95% of the points, and analysing the results.

Code:

```
%% Step 14: Coordinate Normalization and Fundamental Matrix Estimation

% Normalize Points for Camera 1
x_mean = mean(cam1_p2d(1, :));
y_mean = mean(cam1_p2d(2, :));
translated_points = cam1_p2d - [x_mean; y_mean];
Std_x = std(translated_points(1, :));
Std_y = std(translated_points(2, :));
scaled_points_cam1 = [translated_points(1, :) / Std_x; translated_points(2, :) / Std_y];

% T1 Transformation Matrix
T1_matrix = [1 / Std_x, 0, -x_mean / Std_x;
             0, 1 / Std_y, -y_mean / Std_y;
             0, 0, 1];

% Convert to Homogeneous Coordinates
Scaled_homo_cam1 = [scaled_points_cam1; ones(1, size(scaled_points_cam1, 2))];
WED_cam1 = T1_matrix * Scaled_homo_cam1;

% Normalize Points for Camera 2
x1_mean = mean(cam2_p2d(1, :));
y1_mean = mean(cam2_p2d(2, :));
translated_points_2 = cam2_p2d - [x1_mean; y1_mean];
Std_x_2 = std(translated_points_2(1, :));
Std_y_2 = std(translated_points_2(2, :));
scaled_points_cam2 = [translated_points_2(1, :) / Std_x_2; translated_points_2(2, :) / Std_y_2];
```

```
% T2 Transformation Matrix
T2_matrix = [1 / Std_x_2, 0, -x1_mean / Std_x_2;
             0, 1 / Std_y_2, -y1_mean / Std_y_2;
             0, 0, 1];

% Convert to Homogeneous Coordinates
Scaled_homo_cam2 = [scaled_points_cam2; ones(1, size(scaled_points_cam2,
2))];
WED_cam2 = T2_matrix * Scaled_homo_cam2;

% Add Gaussian Noise
noise_std_dev = 1;
cam1_p2d_14 = WED_cam1(1:2, :) + normrnd(0, noise_std_dev, size(cam1_p2d));
cam2_p2d_14 = WED_cam2(1:2, :) + normrnd(0, noise_std_dev, size(cam2_p2d));
% Fundamental Matrix Estimation without Normalization
U_n = [];
for i = 1:size(cam1_p2d, 2)
    U_row = [
        cam1_p2d(1, i) * cam2_p2d(1, i), ...
        cam1_p2d(2, i) * cam2_p2d(1, i), ...
        cam2_p2d(1, i), ...
        cam1_p2d(1, i) * cam2_p2d(2, i), ...
        cam1_p2d(2, i) * cam2_p2d(2, i), ...
        cam2_p2d(2, i), ...
        cam1_p2d(1, i), cam1_p2d(2, i), 1
    ];
    U_n = [U_n; U_row];
end
% Fundamental Matrix Estimation
U_n_14 = [];
for i = 1:size(cam1_p2d_14, 2)
    U_row = [
        cam1_p2d_14(1, i) * cam2_p2d_14(1, i), ...
        cam1_p2d_14(2, i) * cam2_p2d_14(1, i), ...
        cam2_p2d_14(1, i), ...
        cam1_p2d_14(1, i) * cam2_p2d_14(2, i), ...
        cam1_p2d_14(2, i) * cam2_p2d_14(2, i), ...
        cam2_p2d_14(2, i), ...
        cam1_p2d_14(1, i), cam1_p2d_14(2, i), 1
    ];
    U_n_14 = [U_n_14; U_row];
end

[~, ~, V] = svd(U_n_14);
F_14 = reshape(V(:, end), 3, 3)';
[U, D, V] = svd(F_14);
F_14_noised = U * diag([D(1, 1), D(2, 2), 0]) * V';
F_14_noised = F_14_noised / F_14_noised(end, end);

%Condition Numbers
condition_number_U_n = cond(U_n); % Non-normalized
condition_number_U_n_14 = cond(U_n_14); % Normalized
fprintf('Condition Number_Non-Normalized: %.2f\n', condition_number_U_n);
fprintf('Condition Number_Normalized: %.2f\n', condition_number_U_n_14);

% Sum of Absolute Differences
SAD_noised_14 = sum(abs(F(:) - F_14_noised(:)));
fprintf('Sum of Absolute Differences step14: %.4f\n', SAD_noised_14);

% Epipolar Geometry Visualization
```

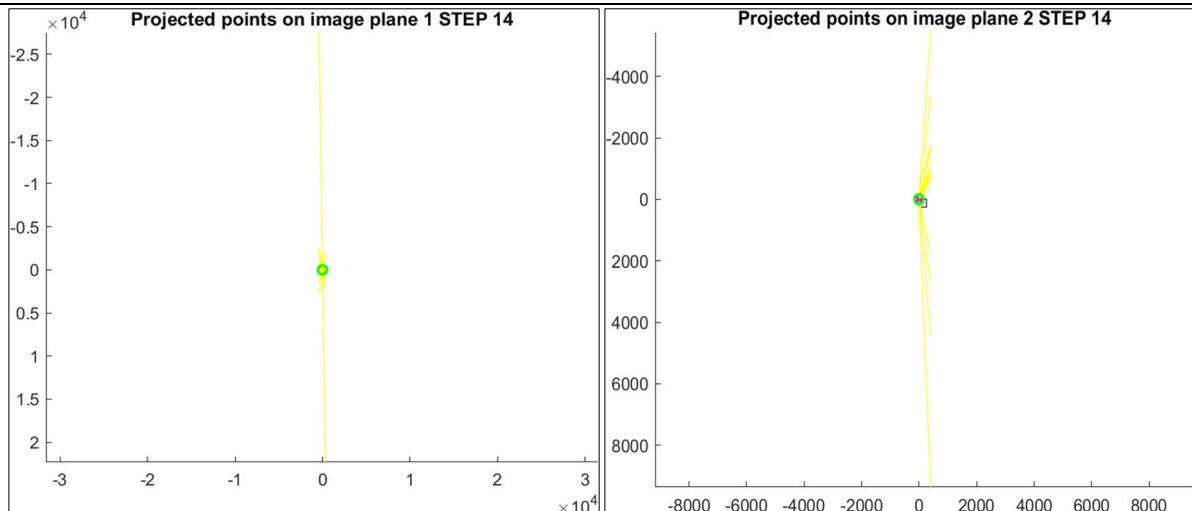
```
cam1_fig = mvg_show_projected_points(cam1_p2d_14, imageSize, 'Projected
points on image plane 1 STEP 14');
cam2_fig = mvg_show_projected_points(cam2_p2d_14, imageSize, 'Projected
points on image plane 2 STEP 14');

[~, ~, c1_l_coeff_s14_noise, c2_l_coeff_s14_noise] =
mvg_compute_epipolar_geom_modif(cam1_p2d_14, cam2_p2d_14, F_14_noised);
[cam1_fig, cam2_fig] = mvg_show_epipolar_lines(cam1_fig, cam2_fig,
c1_l_coeff_s14_noise, c2_l_coeff_s14_noise, [-400, 1; 300, 400], 'y');

% Compute Epipoles
[U, ~, V] = svd(F_14_noised);
ep1_14_noise = V(:, end) / V(end, end);
ep2_14_noise = U(:, end) / U(end, end);
[~, ~] = mvg_show_epipoles(cam1_fig, cam2_fig, ep1_14_noise, ep2_14_noise);
```

Output:

```
Sum of Absolute Differences (F vs Noisy_F13): 0.04487
Condition Number_Non-Normalized: 173696290650720370688.00
Condition Number_Normalized: 63949.89
Sum of Absolute Differences step14: 0.2461
```



3. Conclusion

This lab demonstrated the process of constructing and analysing the Fundamental matrix for stereo vision using both analytical methods and the 8-point algorithm. The results confirmed that in a noise-free scenario, the epipolar geometry was consistent, with epipolar lines converging at the epipoles. In noisy situation deviations from the epipolar lines and misalignment of epipolar lines was observed. By imposing the rank-2 constraint on the Fundamental matrix using SVD, the epipolar lines converged at a single point, validating the corrected Fundamental matrix. Additionally, the normalization of 2D points significantly improved the condition number of the matrix, leading to better numerical stability and more accurate results.

4. Appendix (Code)

```
1. % This a empty script to help you move faster on you lab work.
2. clear all;
3. close all;
```

```

4. clc;
5.
6. %% Step 1
7. % Camera 1
8. au1 = 100; av1 = 120; uo1 = 128; vo1 = 128;
9. imageSize = [256 256];
10.
11.     %% Step 2
12.     % Camera 2
13.     au2 = 90; av2 = 110; uo2 = 128; vo2 = 128;
14.     ax = 0.1; by = pi/4; cz = 0.2; % XYZ EULER
15.     tx = -1000; ty = 190; tz = 230;
16.
17.     %% STEP 3
18.     % Compute intrinsic matrices and projection matrices
19.
20.     K1 = [au1 0 uo1; 0 av1 vo1; 0 0 1]; % Intrinsics matrix for
        camera 1
21.     wR1c = eye(3); % rotation of camera 1, from the camera to the
        world coordinate frame
22.     wt1c = [0 0 0]'; % translation of camera 1, from the camera to
        the world coordinate frame
23.
24.     % Note: ***** You have to add your own code from here
        onward *****
25.     K2 = [au2 0 uo2; 0 av2 vo2; 0 0 1]; %Intrinsic matrix for camera
        2
26.
27.     %computing rotaional matrix for camera 2 from euler angles
28.     Rotx=[1 0 0;0 cos(ax) -sin(ax); 0 sin(ax) cos(ax)];
29.     Roty=[cos(by) 0 sin(by); 0 1 0;-sin(by) 0 cos(by)];
30.     Rotz=[cos(cz) -sin(cz) 0;sin(cz) cos(cz) 0; 0 0 1];
31.     wR2c = Rotx*Roty*Rotz;%combined rotation for camera2
32.     wt2c = [tx;ty;tz];%Translation for camera 2
33.
34.
35.     P1 = K1 * [wR1c' -wR1c' * wt1c];
36.
37.     P2 = K2 * [wR2c' -wR2c' * wt2c]; % Projection matrix for Camera
        2
38.     %% STEP 4
39.
40.     % Attention: This is an invented matrix just to have some input
        for the drawing
41.     % functions. You have to compute it properly
42.     %F = [3e-05 7e-05 -0.006;2e-05 -2.e-05 0.01;-0.009 -0.01 1];
43.     % Define relative rotation from Camera 2 to Camera 1
44.     c1Rc2 = wR1c' * wR2c;
45.
46.     % Define skew-symmetric matrix for translation vector from
        Camera 1 to Camera 2
47.     c1tc2x = [0, -tz, ty; tz, 0, -tx; -ty, tx, 0];
48.
49.     % Compute the fundamental matrix

```

```

50.     F = inv(K2)' * c1Rc2' * c1tc2x * inv(K1);
51.
52.     F = F./F(3,3);
53.
54.     fprintf('Step 4:\n\tAnalytically obtained F:\n');
55.     disp(F);
56.     %% STEP 5
57.     V(:,1) = [100;-400;2000];
58.     V(:,2) = [300;-400;3000];
59.     V(:,3) = [500;-400;4000];
60.     V(:,4) = [700;-400;2000];
61.     V(:,5) = [900;-400;3000];
62.     V(:,6) = [100;-40;4000];
63.     V(:,7) = [300;-40;2000];
64.     V(:,8) = [500;-40;3000];
65.     V(:,9) = [700;-40;4000];
66.     V(:,10) = [900;-40;2000];
67.     V(:,11) = [100;40;3000];
68.     V(:,12) = [300;40;4000];
69.     V(:,13) = [500;40;2000];
70.     V(:,14) = [700;40;3000];
71.     V(:,15) = [900;40;4000];
72.     V(:,16) = [100;400;2000];
73.     V(:,17) = [300;400;3000];
74.     V(:,18) = [500;400;4000];
75.     V(:,19) = [700;400;2000];
76.     V(:,20) = [900;400;3000];
77.
78.     %% STEP 6
79.     % Projection on image planes
80.     cam1_p2d = mvg_projectPointToImagePlane(V,P1);
81.     cam2_p2d = mvg_projectPointToImagePlane(V,P2);
82.
83.     %% STEP 7
84.     % example of the plotting functions
85.     % Draw 2D projections on image planes
86.     cam1_fig =
mvg_show_projected_points(cam1_p2d(1:2,:), imageSize, 'Projected points
on image plane 1 for Analytical F');
87.     cam2_fig =
mvg_show_projected_points(cam2_p2d(1:2,:), imageSize, 'Projected points
on image plane 2 for Analytical F');
88.
89.     % Draw epipolar lines
90.     [~,~,c1_l_coeff,c2_l_coeff] =
mvg_compute_epipolar_geom_modif(cam1_p2d,cam2_p2d,F);
91.     [cam1_fig,cam2_fig] = mvg_show_epipolar_lines(cam1_fig,
cam2_fig, c1_l_coeff,c2_l_coeff, [-400,1;300,400], 'b');
92.
93.     % Draw epipoles
94.     % Draw epipoles Using Null space
95.     e1 = null(F);
96.     e2 = null(F');
97.     % Normalize

```

```

98.      ep_1 = e1/e1(3);
99.      ep_2 = e2/e2(3);
100.     [~,~] = mvg_show_epipoles(cam1_fig, cam2_fig,ep_1,ep_2);
101.
102.     %% Step8
103.     %Fundamental matrix with 8-points
104.     enforce_rank2 = 0;
105.     New_F = compute_fundamental_m(cam1_p2d,cam2_p2d);
106.     New_F = New_F./New_F(3,3);%normalizing
107.     fprintf('Fundamental_with_8_point:Step8:\n');
108.     disp(New_F);
109.
110.     %% Step9
111.     % sum of absolute differencees
112.     sad= sum(abs(F(:) - New_F(:)));
113.     disp(['Sum of Absolute Differences of F ana New_F: ',
        num2str(sad)]);
114.
115.
116.     %% Step10
117.     % Display projected points on image planes
118.     img1_proj = mvg_show_projected_points(cam1_p2d(1:2, :),
        imageSize, 'Projected points on Image Plane 1 (8-Point, STEP10)');
119.     img2_proj = mvg_show_projected_points(cam2_p2d(1:2, :),
        imageSize, 'Projected points on Image Plane 2 (8-Point, STEP10)');
120.
121.     % Compute and draw epipolar lines
122.     [~, ~, epilines_coeffs_img1, epilines_coeffs_img2] =
        mvg_compute_epipolar_geom_modif(cam1_p2d, cam2_p2d, New_F);
123.     [img1_with_lines, img2_with_lines] =
        mvg_show_epipolar_lines(img1_proj, img2_proj, epilines_coeffs_img1,
        epilines_coeffs_img2, [-400, 1; 300, 400], 'k');
124.
125.     % Compute epipoles
126.     epipole_img1 = null(New_F);
127.     epipole_img2 = null(New_F');
128.
129.     % Normalize epipoles
130.     epipole_img1 = epipole_img1 / epipole_img1(3);
131.     epipole_img2 = epipole_img2 / epipole_img2(3);
132.
133.     % Show epipoles on images
134.     [~, ~] = mvg_show_epipoles(img1_with_lines, img2_with_lines,
        epipole_img1, epipole_img2);
135.
136.
137.     %% Step 11: Adding noise to points
138.     % standard deviation for noise
139.     noise_std_dev = 0.5;
140.
141.     % Generate random noise with normal distribution
142.     cam1_points_noisy = cam1_p2d + normrnd(0, noise_std_dev,
        size(cam1_p2d));

```

```

143.     cam2_points_noisy = cam2_p2d + normrnd(0, noise_std_dev,
        size(cam2_p2d));
144.
145.     % Displaying the original and noisy points for comparison
146.     fprintf('Step 11: Noise-free points (Camera 1):\n');
147.     disp(cam1_p2d);
148.
149.     fprintf('Step 11: Noisy points (Camera 1):\n');
150.     disp(cam1_points_noisy);
151.
152.
153.     %% Step12
154.     %Step8 with noisy points
155.     % Fundamental with 8 point for noisy Points
156.     enforce_rank2 = 0;
157.     Noisy_F12 =
        compute_fundamental_m(cam1_points_noisy, cam2_points_noisy);
158.     % normalize all the points by
159.     Noisy_F12 = Noisy_F12./Noisy_F12(end, end);
160.     fprintf('Step 12: Fundamental_M_for_noise_point:\n');
161.     disp(Noisy_F12);
162.
163.     %Step9 for noisy
164.     % error difference with noise
165.     % sum of absolute differences(SAD)
166.     sad_noisy = sum(abs(F(:) - Noisy_F12(:)));
167.     disp(['Sum of Absolute Differences of F ana Noisy_F: ',
        num2str(sad_noisy)]);
168.
169.     %Step10 for noisy
170.     % Drawing Epipolar lines and epipole for noised one
171.     cam1_fig =
        mvg_show_projected_points(cam1_points_noisy(1:2,:), imageSize, 'Project
        ed points on image plane 1 for noisy Step12');
172.     cam2_fig =
        mvg_show_projected_points(cam2_points_noisy(1:2,:), imageSize, 'Project
        ed points on image plane 2 for noisy Step12');
173.
174.     % Draw epipolar lines
175.     [~,~,c1_l_coeff_s12_noise,c2_l_coeff_s12_noise] =
        mvg_compute_epipolar_geom_modif(cam1_points_noisy, cam2_points_noisy, N
        oisy_F12);
176.     [cam1_fig, cam2_fig] = mvg_show_epipolar_lines(cam1_fig,
        cam2_fig, c1_l_coeff_s12_noise, c2_l_coeff_s12_noise, [-
        400,1;300,400], 'c');
177.
178.     % Draw epipoles
179.     % Epipoles Investigation with SVD
180.     [U,~, V] = svd(Noisy_F12);
181.     ep1_12_noise= V(:, end);
182.     ep2_12_noise= U(:, end);
183.
184.     % Normalization
185.     ep1_12_noise = ep1_12_noise / ep1_12_noise(3);

```

```

186.     ep2_12_noise = ep2_12_noise / ep2_12_noise(3);
187.
188.     [~,~] = mvg_show_epipoles(cam1_fig,
    cam2_fig,ep1_12_noise,ep2_12_noise);
189.
190.
191.     %% Step 13: Adding Noise and Recomputing
192.
193.     % Add Gaussian noise to points (std = 1 for 95% of values in [-
    2, +2])
194.     noise_std_dev = 1;
195.     cam1_points_noisy_13 = cam1_p2d + normrnd(0, noise_std_dev,
    size(cam1_p2d));
196.     cam2_points_noisy_13 = cam2_p2d + normrnd(0, noise_std_dev,
    size(cam2_p2d));
197.
198.     % Compute fundamental matrix with noisy points and normalize
199.     Noisy_F13 = compute_fundamental_m(cam1_points_noisy_13,
    cam2_points_noisy_13);
200.     Noisy_F13 = Noisy_F13 ./ Noisy_F13(end, end);
201.     fprintf('Step 13: Fundamental Matrix with Noisy Points:\n');
202.     disp(Noisy_F13);
203.
204.     % Compute error (Sum of Absolute Differences) between F and
    noisy F
205.     sad_noisy_13 = sum(abs(F(:) - Noisy_F13(:)));
206.     disp(['Sum of Absolute Differences (F vs Noisy_F13): ',
    num2str(sad_noisy_13)]);
207.
208.     % Visualize noisy points projection on image planes
209.     cam1_fig = mvg_show_projected_points(cam1_points_noisy_13(1:2,
    :), imageSize, 'Image 1: Noisy Points Step13');
210.     cam2_fig = mvg_show_projected_points(cam2_points_noisy_13(1:2,
    :), imageSize, 'Image 2: Noisy Points Step13');
211.
212.     % Compute and draw epipolar lines with noisy points
213.     [~, ~, c1_l_coeff_s13_noise, c2_l_coeff_s13_noise] =
    mvg_compute_epipolar_geom_modif(cam1_points_noisy_13,
    cam2_points_noisy_13, Noisy_F13);
214.     [cam1_fig, cam2_fig] = mvg_show_epipolar_lines(cam1_fig,
    cam2_fig, c1_l_coeff_s13_noise, c2_l_coeff_s13_noise, [-400, 1; 300,
    400], 'r');
215.
216.     % Compute epipoles using SVD and normalize
217.     [U, ~, V] = svd(Noisy_F13);
218.     ep1_13_noise = V(:, end) / V(3, end);
219.     ep2_13_noise = U(:, end) / U(3, end);
220.
221.     % Display epipoles on images
222.     [~, ~] = mvg_show_epipoles(cam1_fig, cam2_fig, ep1_13_noise,
    ep2_13_noise);
223.
224.     %% Step 14: Coordinate Normalization and Fundamental Matrix
    Estimation
225.

```

```

226.     % Normalize Points for Camera 1
227.     x_mean = mean(cam1_p2d(1, :));
228.     y_mean = mean(cam1_p2d(2, :));
229.     translated_points = cam1_p2d - [x_mean; y_mean];
230.     Std_x = std(translated_points(1, :));
231.     Std_y = std(translated_points(2, :));
232.     scaled_points_cam1 = [translated_points(1, :) / Std_x;
        translated_points(2, :) / Std_y];
233.
234.     % T1 Transformation Matrix
235.     T1_matrix = [1 / Std_x, 0, -x_mean / Std_x;
236.                 0, 1 / Std_y, -y_mean / Std_y;
237.                 0, 0, 1];
238.
239.     % Convert to Homogeneous Coordinates
240.     Scaled_homo_cam1 = [scaled_points_cam1; ones(1,
        size(scaled_points_cam1, 2))];
241.     WED_cam1 = T1_matrix * Scaled_homo_cam1;
242.
243.     % Normalize Points for Camera 2
244.     x1_mean = mean(cam2_p2d(1, :));
245.     y1_mean = mean(cam2_p2d(2, :));
246.     translated_points_2 = cam2_p2d - [x1_mean; y1_mean];
247.     Std_x_2 = std(translated_points_2(1, :));
248.     Std_y_2 = std(translated_points_2(2, :));
249.     scaled_points_cam2 = [translated_points_2(1, :) / Std_x_2;
        translated_points_2(2, :) / Std_y_2];
250.
251.     % T2 Transformation Matrix
252.     T2_matrix = [1 / Std_x_2, 0, -x1_mean / Std_x_2;
253.                 0, 1 / Std_y_2, -y1_mean / Std_y_2;
254.                 0, 0, 1];
255.
256.     % Convert to Homogeneous Coordinates
257.     Scaled_homo_cam2 = [scaled_points_cam2; ones(1,
        size(scaled_points_cam2, 2))];
258.     WED_cam2 = T2_matrix * Scaled_homo_cam2;
259.
260.
261.     % Add Gaussian Noise
262.     noise_std_dev = 1;
263.     cam1_p2d_14 = WED_cam1(1:2, :) + normrnd(0, noise_std_dev,
        size(cam1_p2d));
264.     cam2_p2d_14 = WED_cam2(1:2, :) + normrnd(0, noise_std_dev,
        size(cam2_p2d));
265.     % Fundamental Matrix Estimation without Normalization
266.     U_n = [];
267.     for i = 1:size(cam1_p2d, 2)
268.         U_row = [
269.             cam1_p2d(1, i) * cam2_p2d(1, i), ...
270.             cam1_p2d(2, i) * cam2_p2d(1, i), ...
271.             cam2_p2d(1, i), ...
272.             cam1_p2d(1, i) * cam2_p2d(2, i), ...
273.             cam1_p2d(2, i) * cam2_p2d(2, i), ...

```

```

274.         cam2_p2d(2, i), ...
275.         cam1_p2d(1, i), cam1_p2d(2, i), 1
276.     ];
277.     U_n = [U_n; U_row];
278. end
279. % Fundamental Matrix Estimation
280. U_n_14 = [];
281. for i = 1:size(cam1_p2d_14, 2)
282.     U_row = [
283.         cam1_p2d_14(1, i) * cam2_p2d_14(1, i), ...
284.         cam1_p2d_14(2, i) * cam2_p2d_14(1, i), ...
285.         cam2_p2d_14(1, i), ...
286.         cam1_p2d_14(1, i) * cam2_p2d_14(2, i), ...
287.         cam1_p2d_14(2, i) * cam2_p2d_14(2, i), ...
288.         cam2_p2d_14(2, i), ...
289.         cam1_p2d_14(1, i), cam1_p2d_14(2, i), 1
290.     ];
291.     U_n_14 = [U_n_14; U_row];
292. end
293.
294. [~, ~, V] = svd(U_n_14);
295. F_14 = reshape(V(:, end), 3, 3)';
296. [U, D, V] = svd(F_14);
297. F_14_noised = U * diag([D(1, 1), D(2, 2), 0]) * V';
298. F_14_noised = F_14_noised / F_14_noised(end, end);
299.
300. %Condition Numbers
301. condition_number_U_n = cond(U_n); % Non-normalized
302. condition_number_U_n_14 = cond(U_n_14); % Normalized
303. fprintf('Condition Number_Non-Normalized: %.2f\n',
304.     condition_number_U_n);
304. fprintf('Condition Number_Normalized: %.2f\n',
305.     condition_number_U_n_14);
305.
306. % Sum of Absolute Differences
307. SAD_noised_14 = sum(abs(F(:) - F_14_noised(:)));
308. fprintf('Sum of Absolute Differences step14: %.4f\n',
309.     SAD_noised_14);
309.
310. % Epipolar Geometry Visualization
311. cam1_fig = mvg_show_projected_points(cam1_p2d_14, imageSize,
312.     'Projected points on image plane 1 STEP 14');
312. cam2_fig = mvg_show_projected_points(cam2_p2d_14, imageSize,
313.     'Projected points on image plane 2 STEP 14');
313.
314. [~, ~, c1_l_coeff_s14_noise, c2_l_coeff_s14_noise] =
315.     mvg_compute_epipolar_geom_modif(cam1_p2d_14, cam2_p2d_14,
316.     F_14_noised);
315. [cam1_fig, cam2_fig] = mvg_show_epipolar_lines(cam1_fig,
316.     cam2_fig, c1_l_coeff_s14_noise, c2_l_coeff_s14_noise, [-400, 1; 300,
317.     400], 'y');
316.
317. % Compute Epipoles
318. [U, ~, V] = svd(F_14_noised);

```

```
319.     ep1_14_noise = V(:, end) / V(end, end);  
320.     ep2_14_noise = U(:, end) / U(end, end);  
321.     [~, ~] = mvq_show_epipoles(cam1_fig, cam2_fig, ep1_14_noise,  
        ep2_14_noise);
```